

Student

I have been writing queries that reference tables directly. But sometimes I want to filter based on a computed value — like 'show me products priced above the average'. Can I nest a query inside another query?

ChatGPT • AI Assistant

CONCEPT CARD

Yes — that is exactly what a **subquery** is. A subquery is a **SELECT** statement nested inside another SQL statement. The outer query uses the result of the inner query.

Three places a subquery can live:

- In the **WHERE** clause — to filter based on a computed value
- In the **FROM** clause — as a derived table (a temporary named result set)
- In the **SELECT** clause — as a scalar subquery that computes one value per row

🔗 ANALOGY

Think of a subquery like a helper function in Python. Your main function calls the helper, gets a result, and uses it. The helper runs first, returns a value, and the main query uses that value to finish its work.

🌐 REAL WORLD APPLICATION

Subqueries appear everywhere in production SQL — ranking queries, conditional aggregations, existence checks, and any time you need to filter on a value that itself requires a query to compute. Understanding when to use a subquery vs a **JOIN** vs a **CTE** separates intermediate developers from senior ones.

Student

What schema are we using?

ChatGPT • AI Assistant

CONCEPT CARD

Same e-commerce schema as previous sections:

- customers(customer_id, name, city, signup_date)
- orders(order_id, customer_id, order_date, total_amount, status)
- products(product_id, name, category, price, stock_qty)

```
SELECT name,
  (SELECT COUNT(*) FROM orders
   WHERE orders.customer_id = c.customer_id) AS order_count
FROM customers c
WHERE customer_id
  IN (SELECT customer_id FROM orders
      WHERE total_amount > 1000)
AND city IN (SELECT city
             FROM (SELECT city, COUNT(*) AS cnt FROM customers
                   GROUP BY city HAVING COUNT(*) > 50) top_cities)
```

1 **Scalar Subquery** — runs once per outer row, returns exactly one value.

2 **WHERE Subquery** — filters outer rows based on inner result set.

3 **Derived Table** — subquery in FROM, given an alias, treated like a real table.

Three Locations Where a Subquery Can Appear

Subquery Types — Quick Reference				
Subquery Type	Location	Returns	Runs How Many Times	Example Use Case
Scalar subquery	SELECT clause	Exactly one value (one row, one column)	Once per outer row	Order count per customer alongside customer data
Uncorrelated subquery	WHERE clause	A value or set of values	Once total — result is reused	Products priced above the catalogue average
Correlated subquery	WHERE or SELECT clause	A value that depends on each outer row	Once per outer row — potentially slow	Customers whose spend exceeds their own city average
Derived table	FROM clause	A full result set with an alias	Once total	Pre-aggregated data used as a source table

Subquery Type	Location	Returns	Runs How Many Times	Example Use Case
EXISTS subquery	WHERE clause	TRUE or FALSE	Once per outer row, short-circuits on first match	Check if a customer has any order at all

sql – Subquery Location Templates – All Three Positions

```
-- 1. Subquery in WHERE (uncorrelated)
SELECT name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products);

-- 2. Subquery in FROM (derived table)
SELECT city, avg_spend
FROM (
    SELECT c.city, AVG(o.total_amount) AS avg_spend
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    GROUP BY c.city
) city_stats
WHERE avg_spend > 1000;

-- 3. Scalar subquery in SELECT
SELECT c.name,
       (SELECT COUNT(*)
        FROM orders o
        WHERE o.customer_id = c.customer_id) AS order_count
FROM customers c;
```

UNCORRELATED SUBQUERIES IN WHERE — THE INNER QUERY RUNS ONCE



ChatGPT • AI Assistant

A two-stage execution diagram. Stage 1 (top box, orange background, label 'Step 1 — Inner Query Executes Once'): shows the SQL 'SELECT AVG(price) FROM products' with a downward arrow to a result bubble showing the number 1249.50. Stage 2 (bottom box, blue background, label 'Step 2 — Outer Query Uses the Result'): shows the SQL 'SELECT name, price FROM products WHERE price > 1249.50' with the value 1249.50 highlighted in green as a substituted literal. A large bracket on the right spans both stages with label 'The inner query result is a constant — reused for every outer row without re-executing.' Below, a performance note box reads: 'Uncorrelated subquery = O(1) inner executions. Efficient.' Clean two-tone diagram, monospace SQL font, arrow connecting Stage 1 result to Stage 2 condition.

Uncorrelated Subquery — Inner Query Runs Once

sql – Uncorrelated Subqueries – Scalar, IN, ALL, ANY

```
-- Scalar subquery: products above average price
SELECT name, category, price
FROM products
WHERE price > (SELECT AVG(price) FROM products)
ORDER BY price DESC;

-- Subquery with IN: orders from top 3 spending customers
SELECT order_id, customer_id, total_amount, status
FROM orders
WHERE customer_id IN (
    SELECT customer_id
    FROM orders
    GROUP BY customer_id
    ORDER BY SUM(total_amount) DESC
    LIMIT 3
);

-- IN with grouped subquery: products in large categories
SELECT name, category, price
FROM products
WHERE category IN (
    SELECT category
    FROM products
    GROUP BY category
    HAVING COUNT(*) ≥ 10
);

-- ALL: more expensive than every Fitness product
SELECT name, price FROM products
WHERE price > ALL (
    SELECT price FROM products WHERE category = 'Fitness'
);

-- WRONG: = with multi-row subquery
-- WHERE category = (SELECT category FROM products WHERE price > 1000);
```



```
-- ERROR

-- CORRECT: IN with multi-row subquery
SELECT name FROM products
WHERE category IN (
    SELECT DISTINCT category FROM products WHERE price > 1000
);
```

Student

Let us start with the simplest case. How do I find products priced above the average price?

ChatGPT • AI Assistant

CONCEPT CARD

An **uncorrelated subquery** is a subquery whose inner query does NOT reference any column from the outer query. It runs exactly **once**, produces a result, and the outer query uses that result.

sql – Code Example

```
SELECT name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products);
```

Step by step:

1. The inner query `SELECT AVG(price) FROM products` runs first — returns one number, say 1249.50.
2. The outer query becomes `WHERE price > 1249.50` — exactly as if you had written that number directly.

🔗 ANALOGY

An uncorrelated subquery is like asking a colleague 'What is the company average salary?' They tell you once — 80,000. Now you check each employee against that one number. The colleague is asked only once.

EXAMPLE CARD

Example 1 — Basic: Products above average price

sql – Code Example

```
SELECT name,
       category,
       price,
       ROUND((SELECT AVG(price) FROM products), 2) AS catalogue_avg
FROM products
WHERE price > (SELECT AVG(price) FROM products)
ORDER BY price DESC;
```

Output (sample):

code – Code Example

name	category	price	catalogue_avg
Professional Cam	Electronics	4999.00	1249.50
Laptop Pro	Electronics	3499.00	1249.50
Coffee Machine	Kitchen	1599.00	1249.50

Example 2 — Subquery with IN: Orders from the top 3 highest-spending customers

sql – Code Example

```
SELECT order_id, customer_id, total_amount, status
FROM orders
WHERE customer_id IN (
    SELECT customer_id
    FROM orders
    GROUP BY customer_id
    ORDER BY SUM(total_amount) DESC
    LIMIT 3
)
ORDER BY total_amount DESC;
```

The inner query returns a list of 3 customer IDs. The outer query filters orders to only those customers.

Example 3 — Subquery returning multiple values with IN

sql – Code Example

```
-- Products in categories that have at least 10 products
SELECT name, category, price
FROM products
WHERE category IN (
    SELECT category
    FROM products
    GROUP BY category
    HAVING COUNT(*) ≥ 10
)
ORDER BY category, price;
```

Example 4 — Subquery with comparison operators: Orders above the average order value for the same status

sql – Code Example

```
-- Orders that are above the overall average order value
SELECT order_id, total_amount, status
FROM orders
WHERE total_amount > (
    SELECT AVG(total_amount)
    FROM orders
    WHERE status = 'delivered'
)
AND status = 'delivered'
ORDER BY total_amount DESC;
```

Student

Can I use comparison operators other than > with a subquery?

ChatGPT • AI Assistant

Yes — any comparison operator works when the subquery returns exactly one value (scalar): = , ≠ , > , < , ≥ , ≤ . When the subquery returns multiple rows, use IN , NOT IN , ANY , or ALL :

sql – Code Example

```
-- ALL: products more expensive than every product in 'Fitness'
category
SELECT name, price FROM products
```

```
WHERE price > ALL (  
    SELECT price FROM products WHERE category = 'Fitness'  
);  
  
-- ANY: products more expensive than at least one 'Fitness' product  
SELECT name, price FROM products  
WHERE price > ANY (  
    SELECT price FROM products WHERE category = 'Fitness'  
);
```

⚠ WARNING

If a subquery is expected to return one value but returns more than one row, the query throws an error: 'subquery returns more than one row.' Always verify your subquery returns exactly one row when using it with `=`, `>`, `<`.

⚠ COMMON PITFALL / MISTAKE

✗ Subquery returns multiple rows — used with `=` operator:

sql — Code Example

```
SELECT name FROM products  
WHERE category = (  
    SELECT category FROM products WHERE price > 1000  
); -- ERROR if multiple categories match
```

✓ Correct — use `IN` when subquery may return multiple rows:

sql — Code Example

```
SELECT name FROM products  
WHERE category IN (  
    SELECT category FROM products WHERE price > 1000  
);
```

🧑 INTERVIEW CARD

Frequency: High

Question: What is an uncorrelated subquery? How does it differ from a correlated subquery?

Expected Answer: An uncorrelated subquery does not reference any column from the outer query, so it runs exactly once. Its result is computed first and reused for every row of the outer query — like a constant. A correlated subquery references a

column from the outer query, so it must re-execute once for every row of the outer query — making it $O(n)$ in complexity and potentially slow on large tables.

Follow-up: How would you rewrite an uncorrelated subquery as a JOIN? (Answer: Often you can JOIN the table and filter in WHERE or HAVING — but for simple scalar comparisons like 'above average', a subquery is cleaner and equally efficient.)

🔗 PRACTICE QUIZ

1. How many times does an uncorrelated subquery execute?
2. What error occurs if you use = with a subquery that returns 5 rows?
3. When should you use IN instead of = with a subquery?
4. Predict the output count: `SELECT name FROM products WHERE price > (SELECT MAX(price) FROM products);` — how many rows?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. Exactly once — the result is computed and reused for every outer row.
2. 'Subquery returns more than one row' — a runtime error.
3. Use IN when the subquery may return multiple rows (a list of values).
4. Zero rows — no product can be priced higher than the maximum price.

🔗 CHECKPOINT QUESTION

Write a query that finds all orders whose *totalamount* is above the average *totalamount* of cancelled orders. Show *orderid*, *totalamount*, *status*, and a column called *cancelled_avg* showing the average used for comparison.

CORRELATED SUBQUERIES — THE INNER QUERY RUNS ONCE PER OUTER ROW

A side-by-side performance comparison diagram. Left panel (green background, label 'Uncorrelated Subquery'): shows inner query 'SELECT AVG(price) FROM products' with a single execution arrow pointing down to a result bubble '1249.50'. Below, label reads 'Executes ONCE. Result reused for all outer rows.' A green badge reads 'O(1) inner executions.' Right panel (red-orange background, label 'Correlated Subquery'): shows the inner query 'SELECT AVG(p2.price) FROM products p2 WHERE p2.category = p1.category' with multiple execution arrows — one for each outer row — pointing down to different result bubbles: '2149.00 for Electronics', '549.00 for Fitness', '899.00 for Kitchen', and a continuation symbol '... N times total'. A red badge reads 'O(N) inner executions — N = outer row count.' Below both panels, a rule box reads: 'Rewrite correlated subqueries as JOIN + grouped subquery for large tables.' Clean two-panel layout, consistent color coding.

Correlated vs Uncorrelated Subquery — Execution Count Comparison

sql – Correlated Subquery and Its JOIN Rewrite

```
-- Correlated subquery: products above their category average
SELECT p1.name, p1.category, p1.price
FROM products p1
WHERE p1.price > (
    SELECT AVG(p2.price)
    FROM products p2
    WHERE p2.category = p1.category -- correlated: references outer p1
);

-- Same query with category avg shown alongside
SELECT p1.name,
       p1.category,
       p1.price,
       ROUND((SELECT AVG(p2.price)
              FROM products p2
              WHERE p2.category = p1.category), 2) AS category_avg
FROM products p1
WHERE p1.price > (
    SELECT AVG(p2.price)
    FROM products p2
    WHERE p2.category = p1.category
)
ORDER BY p1.category, p1.price DESC;

-- JOIN REWRITE – compute avg once per category, then join
SELECT p.name, p.category, p.price, ROUND(cat_avg.avg_price, 2) AS
category_avg
FROM products p
JOIN (
    SELECT category, AVG(price) AS avg_price
    FROM products
    GROUP BY category
) cat_avg ON p.category = cat_avg.category
WHERE p.price > cat_avg.avg_price
```

```
ORDER BY p.category, p.price DESC;

-- Correlated: customers above their city's avg spend
SELECT c.name, c.city, SUM(o.total_amount) AS total_spend
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name, c.city
HAVING SUM(o.total_amount) > (
    SELECT AVG(o2.total_amount)
    FROM orders o2
    JOIN customers c2 ON o2.customer_id = c2.customer_id
    WHERE c2.city = c.city
);
```

Student

What if the comparison I need depends on each individual row — like, find customers whose total spend is above the average spend for customers in their own city?

ChatGPT • AI Assistant

CONCEPT CARD

That is a **correlated subquery**. The inner query references a column from the outer query — so it cannot run once and reuse the result. It must re-execute **once for every row** the outer query processes.

sql – Code Example

```
SELECT c.name, c.city, SUM(o.total_amount) AS total_spend
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name, c.city
HAVING SUM(o.total_amount) > (
    SELECT AVG(o2.total_amount)
    FROM orders o2
    JOIN customers c2 ON o2.customer_id = c2.customer_id
    WHERE c2.city = c.city -- references outer query's c.city
);
```


Here `c.city` comes from the outer query. For each customer row, the database substitutes that customer's city into the inner query and re-runs it.

🔗 ANALOGY

Now your colleague cannot answer once — you have to ask them a different question for every employee: 'What is the average salary for people in THIS department?' For 500 employees, you ask 500 times. That is a correlated subquery.

EXAMPLE CARD

Example 1 — Correlated subquery in HAVING: Customers above their city average spend

sql — Code Example

```
SELECT c.name,
       c.city,
       SUM(o.total_amount) AS total_spend
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name, c.city
HAVING SUM(o.total_amount) > (
    SELECT AVG(o2.total_amount)
    FROM orders o2
    JOIN customers c2 ON o2.customer_id = c2.customer_id
    WHERE c2.city = c.city
)
ORDER BY total_spend DESC;
```

Example 2 — Correlated subquery in WHERE: Products priced above their own category average

sql — Code Example

```
SELECT p1.name,
       p1.category,
       p1.price,
       ROUND((
           SELECT AVG(p2.price)
           FROM products p2
           WHERE p2.category = p1.category
       ), 2) AS category_avg
FROM products p1
WHERE p1.price > (
    SELECT AVG(p2.price)
    FROM products p2
    WHERE p2.category = p1.category
)
```

```
WHERE p2.category = p1.category -- references outer p1.category
)
ORDER BY p1.category, p1.price DESC;
```

Output (sample):

code – Code Example

name	category	price	category_avg
Professional Cam	Electronics	4999.00	2149.00
Laptop Pro	Electronics	3499.00	2149.00
Premium Yoga Mat	Fitness	899.00	549.00

Student

This sounds slow. If I have 10,000 products, the inner query runs 10,000 times?

ChatGPT • AI Assistant

 WARNING

Exactly right. Correlated subqueries are **O(n)** in inner executions — the inner query runs once per outer row. On large tables this is a serious performance problem.

(

$$\text{Total\;inner\;executions} = N_{\{\text{outer\;rows}\}}$$

For 10,000 products, the category-average subquery runs 10,000 times. Modern query optimizers can sometimes cache or rewrite this automatically — but you should not rely on that.

When does it matter?

- **Small tables (< 10,000 rows):** correlated subqueries are fine — the performance difference is negligible.
- **Large tables (100k+ rows):** rewrite as a JOIN with a grouped subquery or a window function for significant performance gains.

Example 3 — Rewriting a correlated subquery as a JOIN (more efficient)

sql – Code Example

```
-- CORRELATED VERSION (slow on large tables):
SELECT p1.name, p1.category, p1.price
FROM products p1
WHERE p1.price > (
    SELECT AVG(p2.price)
    FROM products p2
    WHERE p2.category = p1.category
);

-- JOIN REWRITE (runs inner query once per category, not once per
product):
SELECT p.name, p.category, p.price
FROM products p
JOIN (
    SELECT category, AVG(price) AS avg_price
    FROM products
    GROUP BY category
) cat_avg ON p.category = cat_avg.category
WHERE p.price > cat_avg.avg_price
ORDER BY p.category, p.price DESC;
```

The JOIN version computes category averages once (one pass over products), then joins — far more efficient than recomputing the average for every product row.

INTERVIEW CARD

Frequency: Very High

Question: What is a correlated subquery and why can it be slow?

Expected Answer: A correlated subquery references a column from the outer query, so it cannot be evaluated independently. It re-executes once for every row the outer query produces — making it $O(n)$ in inner executions. For a table with N rows, the inner query runs N times. On large tables this is slow. The fix is to rewrite it as a JOIN with a grouped subquery: compute the aggregate once per group, join back to the main table, and filter in WHERE. This reduces inner executions from $O(n)$ to $O(\text{distinct groups})$.

Follow-up: When is a correlated subquery acceptable? (Answer: When the table is small, when the logic is significantly clearer as a correlated subquery, or when the query optimizer is known to handle it efficiently — always EXPLAIN ANALYZE first.)

PRACTICE QUIZ

1. How many times does a correlated subquery execute for a 5,000-row outer result?
2. What is the key signal in a subquery that makes it 'correlated'?
3. Rewrite this correlated subquery logic as a JOIN: find orders where *totalamount is above the average totalamount* for their own status.
4. Predict performance: a correlated subquery on a 1-million-row table vs the JOIN rewrite — which is faster and why?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 **ANSWER KEY**

1. 5,000 times — once per outer row.
2. The inner query references a column from the outer query (e.g., `WHERE inner.col = outer.col`).
- 3.

sql – Code Example

```
SELECT o.order_id, o.total_amount, o.status
FROM orders o
JOIN (
    SELECT status, AVG(total_amount) AS avg_amount
    FROM orders GROUP BY status
) s ON o.status = s.status
WHERE o.total_amount > s.avg_amount;
```

1. The JOIN rewrite is dramatically faster — it computes the average once per status (4 executions) vs the correlated subquery which runs 1 million times.

🔍 **CHECKPOINT QUESTION**

You have a products table. Write both the correlated subquery version and the JOIN rewrite version of: 'Find all products whose price is above the average price of products in their own category.' Then explain which version you would use in production and why.

EXISTS AND NOT EXISTS — SHORT-CIRCUIT EXISTENCE CHECKS

A two-panel comparison diagram. Left panel (label 'IN — Full Scan'): shows a subquery result list of 8 customer IDs in a vertical column. An arrow labeled 'Must read entire list before checking membership' points down through all 8 rows. A clock icon at the bottom reads 'Reads all N rows.' Right panel (label 'EXISTS — Short-Circuit'): shows the same 8 customer IDs but with a large green checkmark after row 2 and a stop sign icon. An arrow labeled 'Stops at first match — remaining rows skipped' points to the match at row 2. A speed icon reads 'Stops after first matching row.' Below both panels, a rules box reads: 'EXISTS = presence check only. Always write SELECT 1 inside. NULL-safe. Preferred for large correlated existence checks.' A red warning sub-box: 'NOT IN + NULL = silent zero-row result. Use NOT EXISTS instead.' Clean two-column layout, monospace SQL font for code snippets inside panels.

EXISTS Short-Circuit vs IN Full Scan

sql – EXISTS and NOT EXISTS – Existence Checks

```
-- EXISTS: customers who have at least one order
SELECT c.customer_id, c.name, c.city
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
)
ORDER BY c.name;

-- NOT EXISTS: customers who have never ordered (NULL-safe anti-join)
SELECT c.customer_id, c.name, c.city, c.signup_date
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
)
ORDER BY c.signup_date DESC;

-- EXISTS with condition: customers with at least one delivered order
SELECT c.name, c.city
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
    AND o.status = 'delivered'
);

-- NOT EXISTS: products never ordered
SELECT p.name, p.category, p.price
FROM products p
WHERE NOT EXISTS (
```

```
SELECT 1
FROM order_items oi
WHERE oi.product_id = p.product_id
);

-- NOT IN (DANGEROUS with NULLs) vs NOT EXISTS (safe)
-- DANGEROUS:
-- SELECT name FROM customers WHERE customer_id NOT IN (SELECT
customer_id FROM orders);

-- SAFE:
SELECT c.name FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);
```

Student

Sometimes I just want to check IF a row exists — not what the value is. Is there a better tool than IN for that?

ChatGPT • AI Assistant

CONCEPT CARD

EXISTS tests whether a subquery returns **any row at all**. It returns TRUE if the subquery produces at least one row, FALSE if it produces zero rows. The key advantage: EXISTS **short-circuits** — it stops scanning the moment it finds the first matching row.

sql – Code Example

```
SELECT c.name, c.city
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

ANALOGY

IN is like asking 'Is your name on this list?' — it reads every name on the list. EXISTS is like asking 'Is anyone from your company here?' — you stop as soon as you find one person. Faster when the list is long.

EXAMPLE CARD

Example 1 — EXISTS: Customers who have placed at least one order

sql — Code Example

```
SELECT c.customer_id,  
       c.name,  
       c.city  
FROM customers c  
WHERE EXISTS (  
    SELECT 1          -- SELECT 1, SELECT *, or SELECT any column --  
    doesn't matter  
    FROM orders o  
    WHERE o.customer_id = c.customer_id  
)  
ORDER BY c.name;
```

Why **SELECT 1** ? EXISTS only cares whether any row is returned — not what columns or values are in those rows. Writing **SELECT 1** makes the intent explicit: 'I only care about existence.'

Example 2 — NOT EXISTS: Customers who have NEVER placed an order

sql — Code Example

```
SELECT c.customer_id,  
       c.name,  
       c.city,  
       c.signup_date  
FROM customers c  
WHERE NOT EXISTS (  
    SELECT 1  
    FROM orders o  
    WHERE o.customer_id = c.customer_id  
)  
ORDER BY c.signup_date DESC;
```

This is equivalent to the anti-join pattern from SQL 3.3 (**LEFT JOIN ... WHERE o.id IS NULL**) — choose whichever reads more clearly for your team.

Example 3 — EXISTS with a condition: Customers who have at least one delivered order

sql – Code Example

```
SELECT c.name, c.city
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
        AND o.status = 'delivered'
)
ORDER BY c.name;
```

Example 4 — NOT EXISTS: Products that have never been ordered (alternative to anti-join)

sql – Code Example

```
SELECT p.name, p.category, p.price
FROM products p
WHERE NOT EXISTS (
    SELECT 1
    FROM order_items oi
    WHERE oi.product_id = p.product_id
)
ORDER BY p.category;
```

Student

What is the difference between EXISTS and IN? When do I choose one over the other?

ChatGPT • AI Assistant

Great question — this is a common interview topic.

EXISTS vs IN:

- **IN** retrieves the complete list from the subquery first, then checks membership.
- **EXISTS** checks row by row and stops as soon as a match is found — it never builds the full list.

- **EXISTS** is **NULL-safe** — it does not have the NOT IN + NULL problem from SQL 3.3.
- **IN** is cleaner when comparing against a **small static list** or a **simple uncorrelated subquery**.
- **EXISTS** is preferred for **large correlated existence checks** — its short-circuit behavior gives it an advantage.

⚠ COMMON PITFALL / MISTAKE

✗ **NOT IN breaks silently with NULLs (same issue as SQL 3.3):**

sql – Code Example

```
SELECT name FROM customers
WHERE customer_id NOT IN (
    SELECT customer_id FROM orders
); -- Returns ZERO rows if any order has NULL customer_id
```

✓ **NOT EXISTS is NULL-safe:**

sql – Code Example

```
SELECT name FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

MEMORY TRICK CARD

Remember: **EXISTS cares about presence, not values**. Always write **SELECT 1** inside EXISTS — it signals to both the database and the reader that you are checking existence, not extracting data.

🔗 INTERVIEW CARD

Frequency: High

Question: What is the difference between IN and EXISTS? When would you prefer EXISTS?

Expected Answer: IN retrieves the complete subquery result set and checks membership — it reads all rows. EXISTS checks for any matching row and stops immediately on the first match (short-circuit). EXISTS is preferred when: the subquery is large and you only need to know if any match exists; you are doing a correlated existence check per outer row; or the subquery might return NULLs —

NOT EXISTS handles NULLs correctly while NOT IN does not. IN is fine for small lists or simple uncorrelated comparisons.

Follow-up: Does the column name inside `SELECT 1` matter in EXISTS? (Answer: No — EXISTS only checks whether any row is returned, not what is in those rows. `SELECT 1`, `SELECT *`, `SELECT 'x'` — all equivalent inside EXISTS.)

🔍 PRACTICE QUIZ

1. What does EXISTS return if the subquery produces zero rows?
2. Why is NOT EXISTS safer than NOT IN?
3. In `WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)` — which part makes this a correlated subquery?
4. Predict: `WHERE EXISTS (SELECT 1 FROM products WHERE 1 = 1)` — what does this return for every outer row?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. FALSE — the EXISTS condition fails and the outer row is excluded.
2. NOT IN returns zero rows if the subquery contains any NULL (because `x ≠ NULL` evaluates to NULL, not TRUE). NOT EXISTS checks row by row and is not affected by NULL values in the subquery.
3. `o.customer_id = c.customer_id` — the inner query references `c.customer_id` from the outer query.
4. TRUE for every outer row — `WHERE 1 = 1` always returns rows, so EXISTS always evaluates to TRUE. Every outer row would be included.

🔍 CHECKPOINT QUESTION

Using NOT EXISTS, write a query that finds all products in the 'Electronics' category that have never been included in any order. Compare this to the anti-join version from SQL 3.3 — write both and note which you find more readable.

SUBQUERIES IN FROM — DERIVED TABLES

A three-layer diagram showing how a derived table is computed. Bottom layer (orange background, label 'Inner Query — Derived Table'): shows the SQL 'SELECT customer_id, SUM(total_amount) AS total_spend FROM orders GROUP BY customer_id' with a result mini-table below it showing 3 sample rows: customer_id 101 / total_spend 14200, customer_id 102 / total_spend 3100, customer_id 103 / total_spend 890. Middle layer (blue background, label 'Alias — city_stats'): an arrow points from the inner result table upward to a box labeled 'city_stats' — treated as a real table by the outer query.' Top layer (green background, label 'Outer Query'): shows 'SELECT * FROM city_stats WHERE total_spend > 5000' with a filtered result showing only customer_id 101 / total_spend 14200. A right-side annotation reads: 'Derived table = temporary virtual table. Aliases are visible to the outer query. Solves the WHERE-alias problem.' Clean three-layer stacked diagram, arrows connect each layer.

Derived Table — Subquery in FROM as a Virtual Table

sql — Derived Tables — Filter on Aggregates, Join with Real Tables

```
-- Basic derived table: cities with above-average spend
SELECT city, ROUND(avg_spend, 2) AS avg_order_value
FROM (
    SELECT c.city, AVG(o.total_amount) AS avg_spend
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    GROUP BY c.city
) city_stats
WHERE avg_spend > (SELECT AVG(total_amount) FROM orders)
ORDER BY avg_spend DESC;

-- Derived table solving the WHERE alias problem
SELECT customer_id, total_spend
FROM (
    SELECT customer_id, SUM(total_amount) AS total_spend
    FROM orders
    WHERE status ≠ 'cancelled'
    GROUP BY customer_id
) customer_summary
WHERE total_spend > 5000
ORDER BY total_spend DESC;

-- Derived table joined to a real table
SELECT c.name, c.city, cs.total_spend, cs.order_count
FROM customers c
JOIN (
    SELECT customer_id,
           SUM(total_amount) AS total_spend,
           COUNT(*)          AS order_count
    FROM orders
    GROUP BY customer_id
) cs ON c.customer_id = cs.customer_id
WHERE cs.total_spend > 3000
ORDER BY cs.total_spend DESC;
```

```
-- Category stats derived table
SELECT category, ROUND(avg_price, 2) AS avg_price,
       product_count, total_stock_value
FROM (
    SELECT category,
           AVG(price) AS avg_price,
           COUNT(*) AS product_count,
           SUM(price * stock_qty) AS total_stock_value
    FROM products
    GROUP BY category
) category_stats
WHERE product_count ≥ 5
ORDER BY total_stock_value DESC;
```

Student

You mentioned subqueries in the FROM clause. When would I use that instead of a WHERE subquery?

ChatGPT • AI Assistant

CONCEPT CARD

A **derived table** is a subquery placed in the FROM clause and given an alias. The outer query treats it exactly like a real table — you can filter it, join it, aggregate it, and sort it.

sql – Code Example

```
SELECT *
FROM (
    SELECT city, AVG(total_amount) AS avg_spend
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    GROUP BY city
) city_stats
WHERE avg_spend > 1000;
```

city_stats is the derived table — it does not exist in the database but is computed on the fly.

🔗 ANALOGY

A derived table is like creating a temporary DataFrame in Pandas, then filtering it:

python – Code Example

```
city_stats = orders.merge(customers).groupby('city')
['total_amount'].mean()
result = city_stats[city_stats > 1000]
```

The derived table in SQL is that intermediate DataFrame — computed in memory, used for the next step.

EXAMPLE CARD

Example 1 — Basic derived table: Cities with above-average order spend

sql – Code Example

```
SELECT city,
       ROUND(avg_spend, 2) AS avg_order_value
FROM (
    SELECT c.city,
           AVG(o.total_amount) AS avg_spend
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    GROUP BY c.city
) city_stats
WHERE avg_spend > (
    SELECT AVG(total_amount) FROM orders
)
ORDER BY avg_spend DESC;
```

This uses a derived table AND a subquery in WHERE — both in the same query.

Example 2 — Derived table: Top-spending customers with their rank

sql – Code Example

```
SELECT customer_id,
       total_spend,
       order_count
FROM (
    SELECT customer_id,
           SUM(total_amount) AS total_spend,
           COUNT(*) AS order_count
    FROM orders
```

```
WHERE status ≠ 'cancelled'
GROUP BY customer_id
) customer_summary
WHERE total_spend > 5000
ORDER BY total_spend DESC
LIMIT 20;
```

Without the derived table, you cannot write `WHERE total_spend > 5000` directly — because `total_spend` is a GROUP BY aggregate alias and WHERE cannot reference it (execution order: WHERE runs before SELECT). The derived table wraps the aggregation, making `total_spend` a real column that WHERE can filter.

Student

So derived tables solve the alias-in-WHERE problem from SQL 3.1?

ChatGPT • AI Assistant

Exactly right — that is one of their most common uses:

⚠️ COMMON PITFALL / MISTAKE

✗ Alias in WHERE — fails (SQL 3.1 execution order problem):

sql — Code Example

```
SELECT customer_id,
       SUM(total_amount) AS total_spend
FROM orders
GROUP BY customer_id
WHERE total_spend > 5000; -- ERROR: total_spend alias not visible in
WHERE
```

✓ Fix with HAVING (simpler when possible):

sql — Code Example

```
SELECT customer_id,
       SUM(total_amount) AS total_spend
FROM orders
GROUP BY customer_id
HAVING SUM(total_amount) > 5000;
```

✓ Fix with derived table (more flexible):

sql – Code Example

```
SELECT customer_id, total_spend
FROM (
    SELECT customer_id, SUM(total_amount) AS total_spend
    FROM orders
    GROUP BY customer_id
) customer_summary
WHERE total_spend > 5000;
```

Example 3 — Joining a derived table to a real table

sql – Code Example

```
SELECT c.name,
       c.city,
       cs.total_spend,
       cs.order_count
FROM customers c
JOIN (
    SELECT customer_id,
           SUM(total_amount) AS total_spend,
           COUNT(*)          AS order_count
    FROM orders
    GROUP BY customer_id
) cs ON c.customer_id = cs.customer_id
WHERE cs.total_spend > 3000
ORDER BY cs.total_spend DESC;
```

Here the derived table `cs` is joined to the `customers` table — the join brings in the customer's name and city alongside the aggregated spend data.

Example 4 — Real analyst pattern: Revenue rank per category

sql – Code Example

```
SELECT p.category,
       ROUND(category_stats.avg_price, 2) AS avg_price,
       category_stats.product_count,
       category_stats.total_stock_value
FROM (
    SELECT category,
           AVG(price) AS avg_price,
```

```
COUNT(*) AS product_count,  
SUM(price * stock_qty) AS total_stock_value  
FROM products  
GROUP BY category  
) category_stats  
JOIN products p ON p.category = category_stats.category  
WHERE category_stats.product_count ≥ 5  
ORDER BY category_stats.total_stock_value DESC;
```

INTERVIEW CARD

Frequency: Medium-High

Question: What is a derived table? When would you use a subquery in FROM instead of WHERE?

Expected Answer: A derived table is a subquery in the FROM clause given an alias — treated like a real table by the outer query. Use it when: you need to filter on an aggregate alias (which WHERE cannot do directly due to execution order); you want to pre-aggregate data before joining it to another table; or the logic is cleaner as a named intermediate result. For complex cases with multiple steps, a CTE (WITH clause) is often cleaner than nested derived tables.

Follow-up: What is the difference between a derived table and a CTE? (Answer: Both produce a named temporary result set. CTEs use the WITH keyword, appear before the main query, can be referenced multiple times, and are generally more readable. Derived tables are inline inside the FROM clause and can only be referenced once.)

PRACTICE QUIZ

1. What must a derived table always have that a regular subquery in WHERE does not need?
2. Why is a derived table useful for filtering on aggregate aliases?
3. Rewrite this using a derived table: `SELECT customer_id, SUM(total_amount) AS spend FROM orders GROUP BY customer_id HAVING SUM(total_amount) > 5000;`
4. What is the Pandas equivalent of a derived table?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

ANSWER KEY

1. An alias — every derived table in FROM must be given a name: `FROM ( ... ) alias_name .`
2. Because WHERE runs before SELECT — aggregate aliases don't exist at WHERE time. Wrapping the aggregation in a derived table makes the alias a real column that the outer WHERE can filter.
3. `SELECT customer_id, spend FROM (SELECT customer_id, SUM(total_amount) AS spend FROM orders GROUP BY customer_id) s WHERE spend > 5000;`
4. An intermediate DataFrame: `df2 = df.groupby('customer_id') ['total_amount'].sum().reset_index()` — then filter `df2[df2['total_amount'] > 5000]` .

🔍 CHECKPOINT QUESTION

Write a query using a derived table that shows each customer's name, city, total spend, and order count — but only for customers whose total spend is more than twice the overall average total spend across all customers. Sort by total spend descending.

SCALAR SUBQUERIES IN SELECT AND 10 PRACTICE PROBLEMS

ChatGPT • AI Assistant

A decision tree flowchart. Root node (white box): 'I need to use a subquery.' First decision diamond: 'Do I need to filter based on a computed value or check existence?' Two branches. Left branch labeled 'Filter on a value': second diamond 'Does the inner query reference the outer query?' — left (NO, green): 'Uncorrelated subquery in WHERE. Runs once. Efficient.' — right (YES, orange): 'Correlated subquery. Runs per row. Consider JOIN rewrite for large tables.' Right branch labeled 'Check existence': leads to green box 'Use EXISTS or NOT EXISTS. Write SELECT 1 inside. NULL-safe.' Below these, a third branch from root: 'Do I need the result as a table to join or filter further?' — leads to blue box 'Use derived table in FROM. Give it an alias. Treat as virtual table.' A fourth branch: 'Do I need one value per row as a new column?' — leads to purple box 'Scalar subquery in SELECT. Must return exactly one row — use aggregate.' Bottom rule box (red border): 'When in doubt: prefer JOIN + derived table over correlated subquery for large tables.' Clean decision tree, rounded diamonds, rectangular action boxes, color-coded branches.

Subquery Decision Tree — Which Type to Use?

sql – 10 Practice Problems – Complete Solutions

```
-- Problem 1: Products above average price
SELECT name, category, price FROM products
WHERE price > (SELECT AVG(price) FROM products)
ORDER BY price DESC;

-- Problem 2: Orders from Delhi customers
SELECT order_id, customer_id, total_amount, status FROM orders
WHERE customer_id IN (SELECT customer_id FROM customers WHERE city =
```

```
'Delhi')
ORDER BY total_amount DESC;

-- Problem 3: EXISTS – customers with an order over 2000
SELECT c.name, c.city FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o
    WHERE o.customer_id = c.customer_id AND o.total_amount > 2000
);

-- Problem 4: NOT EXISTS – products never ordered
SELECT p.name, p.category, p.price FROM products p
WHERE NOT EXISTS (
    SELECT 1 FROM order_items oi WHERE oi.product_id = p.product_id
);

-- Problem 5: Derived table – customers with spend > 5000
SELECT c.name, c.city, cs.total_spend
FROM customers c
JOIN (
    SELECT customer_id, SUM(total_amount) AS total_spend
    FROM orders GROUP BY customer_id
) cs ON c.customer_id = cs.customer_id
WHERE cs.total_spend > 5000 ORDER BY cs.total_spend DESC;

-- Problem 6: Scalar subquery in SELECT
SELECT c.customer_id, c.name, c.city,
    (SELECT COUNT(*) FROM orders o WHERE o.customer_id =
c.customer_id) AS order_count,
    (SELECT COALESCE(SUM(o.total_amount), 0) FROM orders o WHERE
o.customer_id = c.customer_id) AS total_spent
FROM customers c ORDER BY total_spent DESC;

-- Problem 7: Correlated – products above category average
SELECT p1.name, p1.category, p1.price FROM products p1
WHERE p1.price > (SELECT AVG(p2.price) FROM products p2 WHERE
p2.category = p1.category)
ORDER BY p1.category, p1.price DESC;

-- Problem 8: JOIN rewrite of Problem 7
SELECT p.name, p.category, p.price
FROM products p
JOIN (SELECT category, AVG(price) AS avg_price FROM products GROUP BY
category) cat_avg
    ON p.category = cat_avg.category
WHERE p.price > cat_avg.avg_price
```

```
ORDER BY p.category, p.price DESC;

-- Problem 9: Cities with avg order value above overall avg
SELECT city, ROUND(avg_order_value, 2) AS avg_order_value
FROM (
    SELECT c.city, AVG(o.total_amount) AS avg_order_value
    FROM customers c JOIN orders o ON c.customer_id = o.customer_id
    WHERE o.status ≠ 'cancelled'
    GROUP BY c.city
) city_stats
WHERE avg_order_value > (SELECT AVG(total_amount) FROM orders WHERE
status ≠ 'cancelled')
ORDER BY avg_order_value DESC;

-- Problem 10: Mumbai 2023 signups with no delivered orders
SELECT c.customer_id, c.name, c.city, c.signup_date
FROM customers c
WHERE c.city = 'Mumbai'
    AND c.signup_date ≥ '2023-01-01' AND c.signup_date < '2024-01-01'
    AND NOT EXISTS (
        SELECT 1 FROM orders o
        WHERE o.customer_id = c.customer_id AND o.status = 'delivered'
    )
ORDER BY c.signup_date;
```

Student

The last type is scalar subqueries in SELECT. What does that look like?

ChatGPT • AI Assistant

CONCEPT CARD

A **scalar subquery** in the SELECT clause computes one value per outer row. It must return exactly one column and one row for each outer row — otherwise the query throws an error.

sql – Code Example

```
SELECT c.name,
       c.city,
       (SELECT COUNT(*)
```

```
FROM orders o
WHERE o.customer_id = c.customer_id) AS order_count
FROM customers c;
```

For each customer row, the inner query counts that customer's orders. The result appears as a new column alongside the customer's data.

 **WARNING**

A scalar subquery in SELECT is technically a correlated subquery — it runs once per outer row. On large tables, this has the same O(n) performance concern. For heavy analytical queries, window functions (`COUNT(*) OVER (PARTITION BY customer_id)`) are a better alternative — but those are an advanced topic.

EXAMPLE CARD

Example 1 — Scalar subquery: Each customer with their order count

sql – Code Example

```
SELECT c.customer_id,
       c.name,
       c.city,
       (SELECT COUNT(*)
        FROM orders o
        WHERE o.customer_id = c.customer_id) AS order_count,
       (SELECT COALESCE(SUM(o.total_amount), 0)
        FROM orders o
        WHERE o.customer_id = c.customer_id) AS total_spent
FROM customers c
ORDER BY total_spent DESC;
```

Output (sample):

code – Code Example

customer_id	name	city	order_count	total_spent
101	Amelia Watson	Mumbai	8	14200.00
102	Rohan Mehta	Delhi	5	9800.00
205	Sara Khan	Hyd	0	0.00

Example 2 — Scalar subquery: Each product with total quantity ever ordered

sql – Code Example

```
SELECT p.product_id,  
       p.name,  
       p.price,  
       (SELECT COALESCE(SUM(oi.quantity), 0)  
        FROM order_items oi  
        WHERE oi.product_id = p.product_id) AS total_qty_ordered  
FROM products p  
ORDER BY total_qty_ordered DESC;
```

Example 3 — When scalar subquery errors: Must return one row

sql – Code Example

```
-- This FAILS if the subquery returns more than one row:  
SELECT name,  
       (SELECT total_amount FROM orders WHERE customer_id =  
        c.customer_id) AS amount  
FROM customers c;  
-- ERROR: subquery returns more than one row (if customer has multiple  
orders)  
  
-- FIX: aggregate to guarantee one row  
SELECT name,  
       (SELECT MAX(total_amount) FROM orders WHERE customer_id =  
        c.customer_id) AS max_order  
FROM customers c;
```

Student

Can we now go through all 10 practice problems to put everything together?

ChatGPT • AI Assistant

CONCEPT CARD

Here are 10 problems covering all subquery types. Attempt each before reading the solution.

Problem 1 — Easy: Uncorrelated scalar — Products priced above the overall average.

sql – Code Example

```
SELECT name, category, price
FROM products
WHERE price > (SELECT AVG(price) FROM products)
ORDER BY price DESC;
```

Problem 2 — Easy: Uncorrelated IN — Orders placed by customers from Delhi.

sql – Code Example

```
SELECT order_id, customer_id, total_amount, status
FROM orders
WHERE customer_id IN (
    SELECT customer_id FROM customers WHERE city = 'Delhi'
)
ORDER BY total_amount DESC;
```

Problem 3 — Easy: EXISTS — Customers who have placed at least one order over ₹2000.

sql – Code Example

```
SELECT c.name, c.city
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
    AND o.total_amount > 2000
)
ORDER BY c.name;
```

Problem 4 — Easy: NOT EXISTS — Products that have never been ordered.

sql – Code Example

```
SELECT p.name, p.category, p.price
FROM products p
WHERE NOT EXISTS (
    SELECT 1
    FROM order_items oi
    WHERE oi.product_id = p.product_id
)
ORDER BY p.category;
```

Problem 5 — Medium: Derived table — Customers whose total spend is above ₹5000, showing name, city, and total spend.

sql – Code Example

```
SELECT c.name, c.city, cs.total_spend
FROM customers c
JOIN (
    SELECT customer_id, SUM(total_amount) AS total_spend
    FROM orders
    GROUP BY customer_id
) cs ON c.customer_id = cs.customer_id
WHERE cs.total_spend > 5000
ORDER BY cs.total_spend DESC;
```

Problem 6 — Medium: Scalar subquery in SELECT — Each customer with their order count and total spend (0 for no orders).

sql – Code Example

```
SELECT c.customer_id,
       c.name,
       c.city,
       (SELECT COUNT(*) FROM orders o
        WHERE o.customer_id = c.customer_id)           AS
order_count,
       (SELECT COALESCE(SUM(o.total_amount), 0)
        FROM orders o
        WHERE o.customer_id = c.customer_id)           AS total_spent
FROM customers c
ORDER BY total_spent DESC;
```

Problem 7 — Medium: Correlated subquery — Products priced above their own category average.

sql – Code Example

```
SELECT p1.name, p1.category, p1.price
FROM products p1
WHERE p1.price > (
    SELECT AVG(p2.price)
    FROM products p2
    WHERE p2.category = p1.category
```

```
)  
ORDER BY p1.category, p1.price DESC;
```

Problem 8 — Medium: JOIN rewrite of Problem 7 (more efficient).

sql – Code Example

```
SELECT p.name, p.category, p.price  
FROM products p  
JOIN (  
    SELECT category, AVG(price) AS avg_price  
    FROM products  
    GROUP BY category  
) cat_avg ON p.category = cat_avg.category  
WHERE p.price > cat_avg.avg_price  
ORDER BY p.category, p.price DESC;
```

Problem 9 — Hard: Derived table + subquery — Cities where the average order value is above the overall average order value across all cities.

sql – Code Example

```
SELECT city, ROUND(avg_order_value, 2) AS avg_order_value  
FROM (  
    SELECT c.city, AVG(o.total_amount) AS avg_order_value  
    FROM customers c  
    JOIN orders o ON c.customer_id = o.customer_id  
    WHERE o.status ≠ 'cancelled'  
    GROUP BY c.city  
) city_stats  
WHERE avg_order_value > (  
    SELECT AVG(total_amount)  
    FROM orders  
    WHERE status ≠ 'cancelled'  
)  
ORDER BY avg_order_value DESC;
```

Problem 10 — Hard: Correlated subquery with NOT EXISTS — Customers from Mumbai who signed up in 2023 and have never placed a delivered order.

sql – Code Example

```
SELECT c.customer_id,  
       c.name,
```



```
        c.city,
        c.signup_date
FROM customers c
WHERE c.city = 'Mumbai'
      AND c.signup_date ≥ '2023-01-01'
      AND c.signup_date < '2024-01-01'
      AND NOT EXISTS (
        SELECT 1
        FROM orders o
        WHERE o.customer_id = c.customer_id
              AND o.status = 'delivered'
      )
ORDER BY c.signup_date;
```

SUMMARY CARD

SQL 1.4 Key Takeaways:

- A subquery is a SELECT nested inside another SQL statement.
- **Uncorrelated subquery** — does not reference the outer query. Runs once. Efficient.
- **Correlated subquery** — references the outer query. Runs once per outer row. $O(n)$ — slow on large tables.
- **Rewrite correlated subqueries as JOIN + grouped subquery** for large tables.
- **EXISTS** — presence check, short-circuits on first match, NULL-safe.
- **NOT EXISTS** — safer than NOT IN when NULLs may be present.
- **Derived table** — subquery in FROM clause with an alias. Solves the WHERE-alias problem. Treated as a virtual table.
- **Scalar subquery in SELECT** — returns one value per outer row. Must return exactly one row — aggregate to guarantee this.
- Rule of thumb: prefer JOINS and derived tables over correlated subqueries for large tables. Use subqueries when the logic is clearer or the table is small.

✧ AI DEEP DIVE PROMPT

Copy this prompt into Claude or ChatGPT for deeper practice:

'Act as a senior SQL mentor. I just learned SQL subqueries — uncorrelated, correlated, EXISTS, NOT EXISTS, derived tables, and scalar subqueries. Give me 10 progressively harder subquery problems using an e-commerce schema with tables: customers(customerid, name, city, signupdate), orders(orderid, customerid, orderdate, totalamount, status), products(productid, name, category, price, stockqty). For each problem: show me the subquery version first, then show me the equivalent

JOIN rewrite, explain when each approach is better, and give me one interview question about it. Do not give me the next problem until I answer.'

🔍 CHECKPOINT QUESTION

Before moving to SQL 1.5 (Set Operations): without running it, identify every problem in this query and write the fully corrected version:

sql – Code Example

```
SELECT name, city, total_spend
FROM customers
WHERE total_spend > 5000
AND EXISTS (
    SELECT total_amount
    FROM orders
    WHERE status = 'delivered'
);
```

🌟 CHAPTER REVIEW & SUMMARY

What You Learned:

Understanding of the core concepts presented in this chapter including sql 1.4 — subqueries, their architectural components, practical use cases, and design paradigms.

Quick Revision Notes:

- Key definitions and nomenclature for the chapter topic.
- Hands-on code execution patterns and setup checklists.
- Analogy-driven visualization of data structures or layered setups.

Core Visualized Components:

- **Three Locations Where a Subquery Can Appear:** A single large SQL query block displayed in monospace font, with three color-cod...